

Reviewer Pack — Discrete Morse Critical-Cell Count Lower Bounds Frege Depth

Entry cac9d682890f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 18:59:33 UTC

Discrete Morse Critical-Cell Count Lower Bounds Frege Depth

Entry ID: cac9d682890f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 18:59:33 UTC

1. Conjecture

Field A (mathematical branch): Discrete Morse Theory (Forman combinatorial topology) **Field B** (complexity object): Tree-like Frege / Resolution proof DAG depth (gateway to EF lower bounds)

Statement:

Let F be an unsatisfiable 3-CNF on n variables, and let π be the resolution refutation produced by a fixed DPLL strategy (JW-OS branching, no learning); π lifts to a Frege proof of $\neg F$ whose depth $d(\pi)$ is the longest antecedent chain. View the proof DAG $G_\pi=(V,E)$ as a 1-dimensional simplicial complex and apply the canonical greedy Forman matching: process vertices in reverse topological order, pairing each unmatched vertex with its lowest-index unmatched incoming edge; let $M(F)$ be the resulting count of critical 1-cells. Conjecture (C): for every unsatisfiable 3-CNF F with $n \leq 40$, $M(F) \geq d(\pi) - \lceil \log_2 |V| \rceil$, and the asymptotic mean over random 3-CNF at ratio 4.5 satisfies $E[M(F)] = \Theta(n / \log n)$; a single instance violating the per-instance inequality refutes C.

Rationale (proposer's reasoning):

Forman's discrete Morse theory compresses simplicial complexes while preserving homotopy type, and critical 1-cells encode homologically essential branching that cannot be matched away. Proof DAGs have a natural 1-complex structure, but discrete Morse theory has essentially never been wired to propositional proof complexity (<5 papers, none on EF/Frege depth). If C holds, every short Frege proof must have a greedy Morse matching that nearly perfectly pairs lines with edges, giving a new combinatorial obstruction to EF compactness for hard tautologies.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 96d5fc71ef6ffab7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{10, 15, 20, 25, 30, 40\}$ with 30 unsat 3-CNFs each (ratio 4.5), conjecture C is SUPPORTED iff (i) per-instance bound $M(F) \geq d(\pi) - \lceil \log_2 |V| \rceil$ holds for $\geq 27/30$ instances at

every n AND zero instances violate across all 180, and (ii) OLS slope of $\log E[M(F)]$ vs $\log(n/\log n)$ lies in $[0.8, 1.2]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -discrete Morse theory resolution proof complexity critical cells-Forman matching proof DAG Frege depth lower bound-Morse matching DPLL refutation simplicial complex proof

Top relevant hits considered: - [http://arxiv.org/abs/2504.15729v2] Strong discrete Morse theory - [http://arxiv.org/abs/0805.2131v2] Functoriality in Morse theory on closed manifolds - [http://arxiv.org/abs/math/0406486v2] The Morse Complex for a Morse Function on a Manifold with Corners - [http://arxiv.org/abs/1801.08709v2] Adaptive Lower Bound for Testing Monotonicity on the Line - [http://arxiv.org/abs/2111.07483v2] Tradeoffs for small-depth Frege proofs - [http://arxiv.org/abs/1912.03013v1] The canonical pairs of bounded depth Frege systems - [http://arxiv.org/abs/1811.04485v1] Computing discrete Morse complexes from simplicial complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
    while len(primes) < count:
        if is_prime(num):
            primes.append(num)
        num += 1
```

```

    return primes

def dpll(cnf, assignment):
    for clause in cnf:
        if all(lit not in assignment or assignment[lit] != val for lit, val in clause):
            return False
    return True

def random_3cnf(n, m):
    cnf = []
    variables = list(range(1, n + 1))
    for _ in range(m):
        literals = random.sample(variables * 2, 3)
        literals[0] *= -1 if random.choice([True, False]) else 1
        literals[1] *= -1 if random.choice([True, False]) else 1
        literals[2] *= -1 if random.choice([True, False]) else 1
        cnf.append(literals)
    return cnf

def resolution(cnf):
    clauses = set(tuple(sorted(clause)) for clause in cnf)
    while True:
        new_clauses = []
        for (a, b) in combinations(clauses, 2):
            if len(a & b) == 1:
                lit = a ^ b
                new_clause = list((set(a) | set(b)) - {lit})
                if not dpll([new_clause], {}):
                    new_clauses.append(tuple(sorted(new_clause)))
        if not new_clauses:
            break
        clauses.update(new_clauses)
    return len(clauses)

def build_dag(cnf, assignment):
    dag = {}
    for clause in cnf:
        for lit in clause:
            if lit not in assignment or assignment[lit] != 1:
                continue
            for other_lit in clause:
                if other_lit == lit:
                    continue
                if other_lit not in dag:
                    dag[other_lit] = []
                dag[other_lit].append(lit)
    return dag

def greedy_morse_matching(dag):
    unmatched_edges = set()
    for node in sorted(dag, key=lambda x: len(dag[x]), reverse=True):
        for neighbor in dag[node]:
            if neighbor not in dag:
                unmatched_edges.add((node, neighbor))
            break
    return len(unmatched_edges)

```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [10, 15, 20, 25, 30, 40]
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = math.floor(4.5 * n)
        unsat_count = 0

        while unsat_count < 30:
            cnf = random_3cnf(n, m)
            assignment = {}
            if not dpll(cnf, assignment):
                dag = build_dag(cnf, assignment)
                d_pi = resolution(cnf)
                M_F = greedy_morse_matching(dag)
                instances_tested += 1
                total_metric_value += M_F

                if M_F < d_pi - math.ceil(math.log2(len(dag))):
                    conjecture_holds = False
                    counterexample = f"n={n}, M(F)={M_F}, d( $\pi$ )={d_pi}"

            unsat_count += 1

    mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0.0
    return {
        "metric_name": "M(F)",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(30)
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results if "metric_value" in result) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all("conjecture_holds" in result and result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction=1")
    elif any("counterexample" in result for result in results):
        first_failing_seed = next(result["seed"] for result in results if "counterexample" in result)
        print(f"RESULT: FALSIFIED counterexample={next(result['counterexample'] for result in results if result['seed'] == first_failing_seed)}")
    else:

```

```
print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92db060e.py", line 133, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92db060e.py", line 105, in run_trial
    if not dpll(cnf, assignment):
           ^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92db060e.py", line 37, in dpll
    if all(lit not in assignment or assignment[lit] != val for lit, val in clause):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92db060e.py", line 37, in <genexpr>
    if all(lit not in assignment or assignment[lit] != val for lit, val in clause):
                                           ^^^^^^^
```

TypeError: cannot unpack non-iterable int object

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` during DPLL execution (clause unpacking failure), so no data was produced and the pre-registered support condition could not be evaluated. | next: Fix the clause representation in the DPLL routine (ensure clauses are iterables of (lit,val) tuples, not bare ints) and rerun the full $n \in \{10, \dots, 40\}$ sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	115404
2	preregistration	claude_max	opus	0	0	7256
3	novelty	claude_max	opus	0	0	3105
4	novelty	claude_max	opus	0	0	8666
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41298
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13331
7	judge	claude_max	opus	0	0	7390

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 196451 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/cac9d682890f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cac9d682890f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cac9d682890f.tar.gz (if generated)